Python Questions on Tuple and Set

```
In [1]: # What is a tuple in Python and how is it different from a list?

        # Tuple:- It is a collection which is ordered, indexed and unchangeable, allows duplicate members.
        # Normal list:- It is also a collection which is ordered, indexed and changeable, allows duplicate members.
        # The only difference is list() are changeable and tuple() are not changeable.

        # How do you create a tuple with a single element?
        mgt = ("Innomatics",)
        print(type(mgt))
        # To create a single/1/1 after the single element it will consider as tuple other wise it will consider as string.

        <class 'tuple'>

        # Is a tuple immutable or mutable? Justify your answer.

        # Tuples are immutable.
        mgt = "sq", "name", "id", "Yaswanth"
        print(mgt[0])
        mgt[0] = "Innomatics"
        # As you see the above example it is giving a error because tuples are immutable.

          Cell In[7], line 6
            print(mgt[0] = "Innomatics")
                         ^
        SyntaxError: expression cannot contain assignment, perhaps you meant "=="?

In [2]: # Write a Python code to convert a list into a tuple.
        mgt = ["sq", "name", "id", "Yaswanth"]
        print(tuple(mgt))

        ('sq', 'name', 'id', 'Yaswanth')

In [3]: # How do you access elements in a tuple?
        mgt = "sq", "name", "id", "Yaswanth"
        print(mgt[0])
        print(mgt[1])
        print(mgt[2:3])
        print(mgt[-1])

        sq
        name
        ('id',)
        Yaswanth

In [4]: # What happens if you try to change the value of a tuple element?

        # It will give a Error immediately!
        mgt = "sq", "name", "id", "Yaswanth"
        mgt[0] = "mr"

        TypeError                                 Traceback (most recent call last)
        Cell In[4], line 5
              3 mgt = "sq", "name", "id", "Yaswanth"
        ----> 5 mgt[0] = "mr"

        TypeError: 'tuple' object does not support item assignment

In [5]: # Can a tuple contain elements of different data types? Explain with example.

        # Yes, Tuples are heterogeneous.
        # Heterogeneous:- Different, 0, 1, 2, 3, 4, 5, True
        print(mgt)

        ("Innomatics", 25, 45.45, True)

In [6]: # How do you check if an item exists in a tuple?
        # By membership operator "in"
        mgt = "sq", "name", "id", "Yaswanth"
        print("id" in mgt)

        True

In [7]: # Write a program to count the number of occurrences of an element in a tuple.
        mgt = "sq", "name", "id", "Yaswanth", "id"
        print(mgt.count("id"))

        2

In [8]: # How do you find the index of an element in a tuple?
        mgt = "sq", "name", "id", "Yaswanth", "id"
        print(mgt.index("Yaswanth"))

        3

In [9]: # Explain slicing in tuples with an example.
        mgt = "sq", "name", "id", "Yaswanth", "id"
        print(type(mgt[0:3:1]))
        print(mgt[0:3:1])
        print(mgt[0:3:2])
        print(mgt[0:3:3])

        <class 'tuple'>
        ('sq', 'name', 'id')
        ('sq', 'id')
        ('sq',)

In [10]: # How do you concatenate two tuples?
         mgt = "sq", "name", "id", "Yaswanth", "id"
         print(mgt + mgt)

         ('sq', 'name', 'id', 'Yaswanth', 'id', 'sq', 'name', 'id', 'Yaswanth', 'id')

In [11]: # Can a tuple be used as a dictionary key? Why or why not?

         # Yes tuples can be used as dictionary key, when they are immutable.
         # Dictionary keys must be hashable, the value cannot change.
         # If a tuple contains any mutable objects, such as lists, dictionaries,other strings, then these can be used as dictionary keys.
         # If tuple contains any mutable objects like list, they are not hashable and cannot be used as dictionary keys.
         # Example 1:
         d = {(1, 2): "Value1", 3: "Value2"}
         d = {[1, 2]: "Value1", 3: "Value2"} # Not Valid
         # So tuples with immutable objects only can be used as dictionary keys.

         TypeError                                 Traceback (most recent call last)
         Cell In[14], line 5
               3 d = {(1, 2): "Value1", 3: "Value2"}
               4 # Example 2:
         ----> 5 d = {[1, 2]: "Value1", 3: "Value2"} # Not Valid

         TypeError: unhashable type: 'list'

In [12]: # What is a nested tuple? Provide an example.
         # Nested tuple:- Means grouping multiple values into a single tuple.
         # We can assign multiple elements/values consisting of different data types, python will consider as a tuple by simply giving in tuple syntax from.

         # Tuple Unpacking:- Extracting the values from a tuple into variables.
         # The number of variables must be equal to values inside the tuple.

         # Compare tuples and lists in terms of performance.
         # Example 1:
         mgt = ("sq", "name", "id", "Yaswanth")
         print(mgt)
         mgt[0] = "mr"
         # Example 2:
         mgt = "sq", "name", "id", "Yaswanth"
         mgt[0] = "mr"
         # As we see the above example the list is able to change the value to modify it but tuple is giving error.
         # Performance wise both are at same as they are immutable.

         ('sq', 'name', 'id', 'Yaswanth')

         TypeError                                 Traceback (most recent call last)
         Cell In[15], line 6
               4 print(mgt)
               5 mgt[0] = "mr"

         TypeError: 'tuple' object does not support item assignment

In [13]: # Create a nested tuple and access its elements.
         mgt = ("sq", "name", "id", "Yaswanth"), ("Innomatics", 25, 45.45, True, "id", "orange")
         print(mgt[0])
         print(mgt[1])
         print(mgt[0][3])

         ('sq', 'name', 'id', 'Yaswanth')
         ('Innomatics', 25, 45.45, True, 'id', 'orange')
         Yaswanth

In [14]: # What is the use of the 'in' operator with tuples?
         mgt = ("sq", "name", "id", "Yaswanth"), ("Innomatics", 25, 45.45, True, "id", "orange")
         print(True in mgt[1])

         True

         # How do you concatenate two tuples?
         mgt = ("Innomatics", "Institute")
         print(mgt + ("excellent",))
         print(mgt*3)

         ('Innomatics', 'Institute', 'excellent')
         ('Innomatics', 'Institute', 'Innomatics', 'Institute', 'Innomatics', 'Institute')

In [15]: # Explain tuple immutability with an example.
         mgt = "sq", "name", "id", "Yaswanth"
         mgt[0] = "mr"

         TypeError                                 Traceback (most recent call last)
         Cell In[19], line 3
               1 # Explain tuple immutability with an example.
               2 mgt = "sq", "name", "id", "Yaswanth"
         ----> 3 mgt[0] = "mr"

         TypeError: 'tuple' object does not support item assignment

In [16]: # Write a program to find the length of a tuple.
         mgt = ("sq", "name", "id", "Yaswanth"), ("Innomatics", 25, 45.45, True, "id", "orange")
         print(len(mgt))

         2

In [17]: # What is the use of the 'in' operator with tuples?
         mgt = ("sq", "name", "id", "Yaswanth"), ("Innomatics", 25, 45.45, True, "id", "orange")
         print(True in mgt[1])

         True

         # How do you concatenate two tuples?
         mgt = ("Innomatics", "Institute")
         print(mgt + ("excellent",))
         print(mgt*3)

         # Explain tuple immutability with an example.
         mgt = "sq", "name", "id", "Yaswanth"
         mgt[0] = "mr"

         TypeError                                 Traceback (most recent call last)
         Cell In[19], line 3
               1 # Explain tuple immutability with an example.
               2 mgt = "sq", "name", "id", "Yaswanth"
         ----> 3 mgt[0] = "mr"

         TypeError: 'tuple' object does not support item assignment

In [18]: # Write a program to find the length of a tuple.
         mgt = ("sq", "name", "id", "Yaswanth"), ("Innomatics", 25, 45.45, True, "id", "orange")
         print(len(mgt))

         2

In [19]: # What is the use of the 'in' operator with tuples?
         mgt = ("sq", "name", "id", "Yaswanth"), ("Innomatics", 25, 45.45, True, "id", "orange")
         print(True in mgt[1])

         True

         # How do you concatenate two tuples?
         mgt = ("Innomatics", "Institute")
         print(mgt + ("excellent",))
         print(mgt*3)

         # Explain tuple immutability with an example.
         mgt = "sq", "name", "id", "Yaswanth"
         mgt[0] = "mr"

         TypeError                                 Traceback (most recent call last)
         Cell In[19], line 3
               1 # Explain tuple immutability with an example.
               2 mgt = "sq", "name", "id", "Yaswanth"
         ----> 3 mgt[0] = "mr"

         TypeError: 'tuple' object does not support item assignment

In [20]: mgt = "sq", "name", "id", "Yaswanth", 25, 45.45, True, "id", "orange"
         print(mgt.count("id"))
         print(mgt.index("Yaswanth"))
         print(mgt[2:5:1])
         print(mgt[-1])
         print(mgt[0:3:2])
         print(mgt[0:3:3])
         print(mgt[::-1])
         print(mgt[::])
         print(mgt[::2])
         print(mgt[::3])
         print(mgt[::-2])
         print(mgt[::-3])

         2
         3
         ('id', 'Yaswanth', 25)
         orange
         ('sq', 'id')
         ('sq',)
         ('orange', 'id', True, 45.45, 25, 'Yaswanth', 'id', 'name', 'sq')
         ('sq', 'name', 'id', 'Yaswanth', 25, 45.45, True, 'id', 'orange')
         ('sq', 'id', 25, True, 'orange')
         ('sq', 'Yaswanth')
         ('orange', True, 'Yaswanth', 'sq')
         ('orange', 45.45, 'id', 'sq')

In [21]: # Write a code to iterate over a tuple using a for loop.
         mgt = ("sq", "name", "id", "Yaswanth"), ("Innomatics", 25, 45.45, True, "id", "orange")
         for char in mgt:
             print(char)

         sq
         name
         id
         Yaswanth
         ('Innomatics', 25, 45.45, True, 'id', 'orange')

In [22]: # What are the built-in methods available for tuples?
         # Tuples are immutable.
         # If there are tuples built-in methods available for tuples!

         ['__add__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__getnewargs__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'count', 'index']

In [23]: # Demonstrate the use of the 'index()' method with tuples.
         mgt = ("sq", "name", "id", "Yaswanth"), ("Innomatics", 25, 45.45, True, "id", "orange")
         print(mgt[0].index("Yaswanth"))

         3

In [24]: # Can a tuple be used as a dictionary key? Why or why not?

         # Yes tuples can be used as dictionary key, when they are immutable.
         # Dictionary keys must be hashable, the value cannot change.
         # If a tuple contains any mutable objects, such as lists, dictionaries,other strings, then these can be used as dictionary keys.
         # If tuple contains any mutable objects like list, they are not hashable and cannot be used as dictionary keys.
         # Example 1:
         d = {(1, 2): "Value1", 3: "Value2"}
         d = {[1, 2]: "Value1", 3: "Value2"} # Not Valid
         # So tuples with immutable objects only can be used as dictionary keys.

         TypeError                                 Traceback (most recent call last)
         Cell In[14], line 5
               3 d = {(1, 2): "Value1", 3: "Value2"}
               4 # Example 2:
         ----> 5 d = {[1, 2]: "Value1", 3: "Value2"} # Not Valid

         TypeError: unhashable type: 'list'

In [25]: # What is tuple packing and unpacking?
         # Tuple packing:- Means grouping multiple values into a single tuple!
         # We can assign multiple elements/values consisting of different data types, python will consider as a tuple by simply giving in tuple syntax from.

         # Tuple Unpacking:- Extracting the values from a tuple into variables.
         # The number of variables must be equal to values inside the tuple.

         # Compare tuples and lists in terms of performance.
         # Example 1:
         mgt = ("sq", "name", "id", "Yaswanth")
         print(mgt)
         mgt[0] = "mr"
         # Example 2:
         mgt = "sq", "name", "id", "Yaswanth"
         mgt[0] = "mr"
         # As we see the above example the list is able to change the value to modify it but tuple is giving error.
         # Performance wise both are at same as they are immutable.

         ('sq', 'name', 'id', 'Yaswanth')

         TypeError                                 Traceback (most recent call last)
         Cell In[15], line 6
               4 print(mgt)
               5 mgt[0] = "mr"

         TypeError: 'tuple' object does not support item assignment

In [26]: # Create a nested tuple and access its elements.
         mgt = ("sq", "name", "id", "Yaswanth"), ("Innomatics", 25, 45.45, True, "id", "orange")
         print(mgt[0])
         print(mgt[1])
         print(mgt[0][3])

         ('sq', 'name', 'id', 'Yaswanth')
         ('Innomatics', 25, 45.45, True, 'id', 'orange')
         Yaswanth

In [27]: # What is the use of the 'in' operator with tuples?
         mgt = ("sq", "name", "id", "Yaswanth"), ("Innomatics", 25, 45.45, True, "id", "orange")
         print(True in mgt[1])

         True

         # How do you concatenate two tuples?
         mgt = ("Innomatics", "Institute")
         print(mgt + ("excellent",))
         print(mgt*3)

         # Explain tuple immutability with an example.
         mgt = "sq", "name", "id", "Yaswanth"
         mgt[0] = "mr"

         TypeError                                 Traceback (most recent call last)
         Cell In[19], line 3
               1 # Explain tuple immutability with an example.
               2 mgt = "sq", "name", "id", "Yaswanth"
         ----> 3 mgt[0] = "mr"

         TypeError: 'tuple' object does not support item assignment

In [28]: # What is the use of the 'in' operator with tuples?
         mgt = ("sq", "name", "id", "Yaswanth"), ("Innomatics", 25, 45.45, True, "id", "orange")
         print(True in mgt[1])

         True

         # How do you concatenate two tuples?
         mgt = ("Innomatics", "Institute")
         print(mgt + ("excellent",))
         print(mgt*3)

         # Explain tuple immutability with an example.
         mgt = "sq", "name", "id", "Yaswanth"
         mgt[0] = "mr"

         TypeError                                 Traceback (most recent call last)
         Cell In[19], line 3
               1 # Explain tuple immutability with an example.
               2 mgt = "sq", "name", "id", "Yaswanth"
         ----> 3 mgt[0] = "mr"

         TypeError: 'tuple' object does not support item assignment

In [29]: mgt = "sq", "name", "id", "Yaswanth", 25, 45.45, True, "id", "orange"
         print(mgt.count("id"))
         print(mgt.index("Yaswanth"))
         print(mgt[2:5:1])
         print(mgt[-1])
         print(mgt[0:3:2])
         print(mgt[0:3:3])
         print(mgt[::-1])
         print(mgt[::])
         print(mgt[::2])
         print(mgt[::3])
         print(mgt[::-2])
         print(mgt[::-3])

         2
         3
         ('id', 'Yaswanth', 25)
         orange
         ('sq', 'id')
         ('sq',)
         ('orange', 'id', True, 45.45, 25, 'Yaswanth', 'id', 'name', 'sq')
         ('sq', 'name', 'id', 'Yaswanth', 25, 45.45, True, 'id', 'orange')
         ('sq', 'id', 25, True, 'orange')
         ('sq', 'Yaswanth')
         ('orange', True, 'Yaswanth', 'sq')
         ('orange', 45.45, 'id', 'sq')

In [30]: # How do you find the length of a tuple?
         mgt = ("sq", "name", "id", "Yaswanth"), ("Innomatics", 25, 45.45, True, "id", "orange")
         print(len(mgt))

         2

In [31]: # Can a set contain mixed data types? Give example.
         # Yes, Sets are heterogeneous, Sets can be used to perform operations with having different data types.
         mgt_set = {25, 35235, 243, 367, "Yaya", 56.765}
         mgt_set

         {243, 367, 35235, 56.765, 'Yaya', 25}

In [32]: # How do sets help in removing duplicates from a list?

         # Sets automatically removes duplicates because they only store unique elements.
         # By converting a list to a set, duplicates are removed, and then we convert it back to a list for unique elements.

         # What will happen if you try to access an elements using index?
         mgt_set = {25, 35235, 243, 367, "Yaya", 56.765}
         print(mgt_set[0])

         TypeError                                 Traceback (most recent call last)
         Cell In[33], line 3
               1 # What will happen if you try to access an elements using index?
               2 mgt_set = {25, 35235, 243, 367, "Yaya", 56.765}
         ----> 3 print(mgt_set[0])

         TypeError: 'set' object is not subscriptable

In [33]: # Demonstrate a scenario where sets can improve performance.

         # What are frozen sets and how are they different from regular sets?
         ab = 0 | 1 | 2 | 3
         ab.add(4)
         ab.add(5)
         ab.add(6)
         print(ab)

         print("Hai")

         fab = frozenset(ab)
         fab.add(7)
         print(fab)

         # The difference is regular sets are able to use methods like add(), some of methods may not work.

         {0, 1, 2, 3, 4, 5, 6}
         Hai

         AttributeError                            Traceback (most recent call last)
         Cell In[35], line 10
               8 print("Hai")
               9 fab = frozenset(ab)
         ---> 10 fab.add(7)
              11 print(fab)

         AttributeError: 'frozenset' object has no attribute 'add'

In [34]: # How do you create a frozen set?
         # By declaring it in a new variable and converting set to frozen set, Example:
         ab = 0 | 1 | 2 | 3
         fab = frozenset(ab)
         fab

         frozenset({0, 1, 2, 3})

In [35]: # Can a set be nested? Explain.

         set1,2,3,{"Science","Ops"},7]
         print(set)

         # 1. Sets cannot be nested directly because their elements must be immutable.
         # 2. And lastly we can nested.

         # 3. You can nested elements with built-in immutable data types.
         # 4. Nested sets can be useful in scenarios where you need to perform set operations with nesting different data types.

         File <ipython-input-36-ed1ed1ed1ed1>:1
           set1,2,3,{"Science","Ops"},7]
                                      ^
         SyntaxError: unmatched ']'

In [36]: # What are frozen sets and how are they different from regular sets?
         # Frozen sets are immutable sets, which means that once they are created, their elements cannot be changed.
         # Regular sets are mutable, meaning that you can add or remove elements after the set is created.

         frozenset({0, 1, 2, 3})

In [37]: # Can you write real-world use cases for sets in Python?

         # The immense of duplicates by using sets is a real-world usecase for sets in python.
         # We can find differences between two data using sets in python.
```

Python Questions on Tuple and Set

```
In [38]: # Write a Python program to find the sum of all elements in a tuple.
         tp = 25, 6, 7, 3, "sum", "set", 234, 3, 5, "sum"

         tp = 0
         for item in tp:
             if is_numeric(item) > 1:
                 tp += item
         print(tp)

         a = set(tp)
         print(type(a))

         <class 'int'>

In [39]: # Write a Python program to find the second largest number in a list using a tuple.
         tp = 25, 78, 56, 234, 75
         tp1 = sorted(tp)
         tp1[-2]
         print(tp1[-2])

         75

In [40]: # Write a program to convert a tuple of strings into a single string.
         tp = "a", "b"
         print("".join(tp))

         ab

In [41]: # Write a program to remove duplicate elements from a list using a set.
         lst = [1,2,3,3,4,4,4,5,5,6,6,7,7,8,8]
         lst = set(lst)
         lst

         {1, 2, 3, 4, 5, 6, 7, 8}

In [42]: # Given two sets, write a program to check if one set is a subset of another.
         ab = 0 | 1 | 2 | 3
         ab = {0, 1, 2}
         print(set1.issubset(ab))

         True

In [43]: # Write a Python function that takes a tuple of numbers and returns a new tuple with only even numbers.

         # Write a program to find all common elements between two sets.
         set1,2,3,{"Science","Ops"},7]
         set2 = {1,2,3,4,5,6,7,8}
         print(set1.intersection(set2))

         {2, 3}

In [44]: # Write a program to count the frequency of each item in a tuple.

         # Write a Python program to create a union, intersection, and difference of two sets entered by the user.
         a = set(input())
         b = set(input())
         print(a.union(b))
         print(a.intersection(b))
         print(a.difference(b))

         1 2 3 4 5 6 7 8 9
         {'1', '2', '3', '4', '5', '6', '7', '8', '9'}
         set()
         {'1', '2', '3', '4', '5', '6', '7', '8', '9'}

In [45]: # Given a tuple of mixed data types, write a program to extract all string values.
         mgt = ("sq", "name", "id", "Yaswanth", 25, 45.45, True, "id", "orange")
         for i in mgt:
             if type(i) == str:
                 print(i)

         sq
         name
         id
         Yaswanth
         id
         orange

In [46]: # Write a Python function that takes a tuple of numbers and returns the sum.

         # Write a program to find all common elements between two sets.
         set1,2,3,{"Science","Ops"},7]
         set2 = {1,2,3,4,5,6,7,8}
         print(set1.intersection(set2))

         {2, 3}

In [47]: # Write a program to count the frequency of each item in a tuple.

         # Write a Python program to create a union, intersection, and difference of two sets entered by the user.
         a = set(input())
         b = set(input())
         print(a.union(b))
         print(a.intersection(b))
         print(a.difference(b))

         1 2 3 4 5 6 7 8 9
         {'1', '2', '3', '4', '5', '6', '7', '8', '9'}
         set()
         {'1', '2', '3', '4', '5', '6', '7', '8', '9'}

In [48]: # Given a tuple of mixed data types, write a program to extract all string values.
         mgt = ("sq", "name", "id", "Yaswanth", 25, 45.45, True, "id", "orange")
         for i in mgt:
             if type(i) == str:
                 print(i)

         sq
         name
         id
         Yaswanth
         id
         orange

In [49]: # Write a Python function that takes a tuple of numbers and returns the sum.

         # Write a program to find all common elements between two sets.
         set1,2,3,{"Science","Ops"},7]
         set2 = {1,2,3,4,5,6,7,8}
         print(set1.intersection(set2))

         {2, 3}

In [50]: # Write a program to count the frequency of each item in a tuple.

         # Write a Python program to create a union, intersection, and difference of two sets entered by the user.
         a = set(input())
         b = set(input())
         print(a.union(b))
         print(a.intersection(b))
         print(a.difference(b))

         1 2 3 4 5 6 7 8 9
         {'1', '2', '3', '4', '5', '6', '7', '8', '9'}
         set()
         {'1', '2', '3', '4', '5', '6', '7', '8', '9'}
```